PORTFOLIO

PORTFOLIO

임찬진

플레이어들에게 행복한 시간을 주고 싶은 개발자

포트폴리오 목차

Project	Page	사용 언어	사용 엔진
S.T.L	**Page 3 - 10**	**C++20**	**D3D12**
└ 논문 발표	**Page 11**	-	-
└ Deferred 렌더러	**Page 12 - 15**	**C++20**	**D3D12**
TheGreatestNature	**Page 16**	-	**UE5**
ASSASSIN VS MUTANT	**Page 17**	-	**UE5**
FlickDom	**Page 18**	**C#**	**Unity 6**

Other Activities

AI 활용 능력	**Page 19 - 21**	-	-
마무리	**Page 22**	-	-

S.T.L

GITHUB VIDEO

Team Project



S.T.L (Slay The Lord) · D3D12 자체 엔진 게임

glTF 파서

외부 라이브러리 없이 메시·재질 직접 파싱

PBR 렌더링

Cook-Torrance BRDF · IBL · ORM 셰이더

성능 최적화

RenderDoc 분석 · 10FPS → 85FPS+

4인 협동 멀티 액션 RPG

S.T.L (Slay The Lord)

파티와 함께 퀘스트를 수행하고 협동 플레이로 보스 몬스터를 공략하는 다크 판타지 게임

개발 인원	3명
담당 역할	glTF 리소스 파싱, 텍스처 URI 해석, MaterialInfo 매핑, D3D12 PBR 렌더링, 성능 최적화
개발 언어	C++20, STL, HLSL
사용 툴	Visual Studio 2022, VS Code, DirectX 12, ImGui, Json, Git, RenderDoc
제작 기간	2025.06.23 ~ 2026.08.09

렌더링 성능 최적화

대규모 에셋 맵 렌더링 프레임 레이트

10 FPS → 85+ FPS

병목 분석

RenderDoc으로 프레임을 캡처해 패스별 드로우 호출 수와 GPU 소요 시간을 확인하고, 병목이 CPU와 GPU 중 어디에서 발생하는지 구분했습니다.

1	**프레임 캡처**	RenderDoc으로 대규모 에셋 맵 프레임 캡처
2	**원인 확인**	같은 메시의 반복 드로우 호출과 그림자 패스 비용 확인
3	**개선 적용**	Object · Shadow Instancing, Shadow Culling 적용
4	**결과**	10FPS → 85FPS 이상



다수의 에셋이 배치된 S.T.L 맵

Object Instancing

같은 메시를 묶어 한 번의 드로우 호출로 렌더링

Shadow Instancing

그림자 패스에도 인스턴싱을 적용해 호출 수 감소

Shadow Culling

그림자에 영향을 주지 않는 객체를 그림자 렌더링에서 제외

보스 공략·전투 판정 등 게임 콘텐츠는 팀원이 구현했으며, 저는 그래픽스 파트(리소스 파이프라인 · PBR 렌더링 · 성능 최적화)를 담당했습니다.

월드 구성 렌더링

Unreal → 자체 엔진

Unreal 배치 데이터 기반 맵 구성

Unreal에서 구성한 Mesh와 Transform 데이터를 자체 엔진의 월드 좌표계로 변환해 같은 맵을 구성했습니다.

1	**glTF 파싱**	Mesh Primitive와 Material 데이터를 파싱
2	**좌표 변환**	Transform 값을 엔진 좌표와 스케일 기준으로 변환
3	**월드 배치**	메쉬를 월드 위치에 배치해 렌더링



Unreal 배치 데이터로 구성한 S.T.L 마을 맵

Unreal 레벨 배치 → glTF 추출 → Mesh · Material 파싱 → 좌표 · 스케일 변환 → 자체 엔진 월드 배치

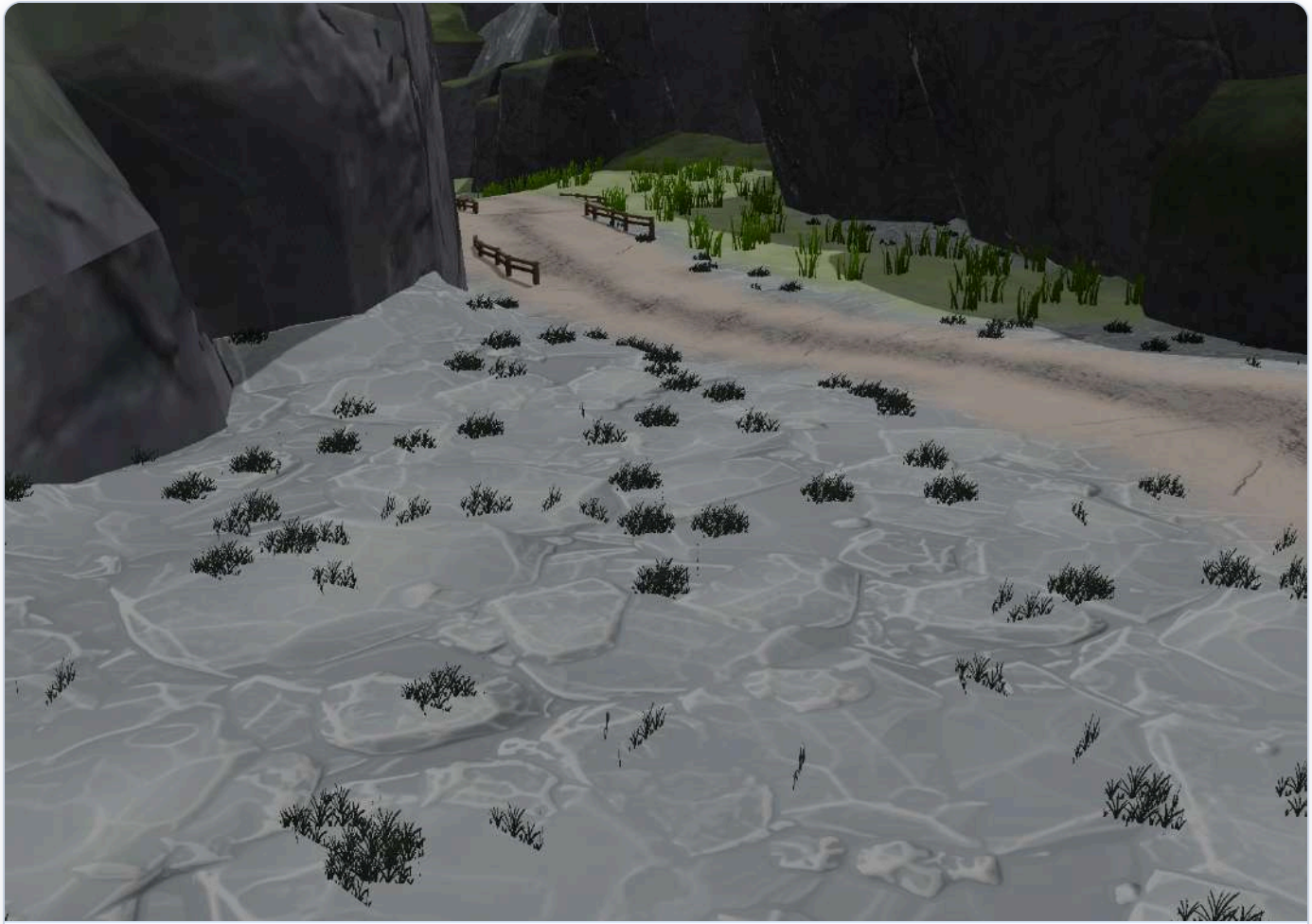
Landscape 텍스처 매핑

Mesh export 없이

raw 파일만으로 Landscape 재구성

Landscape의 높이 정보와 타일별 텍스처 레이어 정보를 raw 파일로 추출해, 자체 엔진에서 같은 지형을 다시 구성했습니다.

1	**데이터 변환**	raw 값을 지형 타일과 텍스처 레이어 데이터로 변환
2	**텍스처 선택**	타일별 raw 값에 맞춰 지형 텍스처 선택
3	**지형 렌더링**	텍스처 레이어를 자체 엔진 지형 렌더링에 적용



raw 파일로 재구성한 지형과 텍스처 레이어

glTF PBR 코드 구현

glTF Material → PBR Shader

Material 데이터를 셰이더까지

glTF Material 데이터를 엔진용 MaterialInfo로 변환하고 D3D12 바인딩을 통해 PBR 셰이더로 전달했습니다.

1	**URI 해석**	images / textures 인덱스를 따라 텍스처 URI를 해석
2	**경로 복원**	base_path와 URI를 조합해 dds 텍스처 경로 복원
3	**셰이더 바인딩**	Material 상수와 SRV를 PBR 셰이더에 바인딩

01
URI Resolve

images / textures 인덱스 기반 경로 해석

material이 참조하는 텍스처 URI를 glTF 인덱스 순서대로 해석했습니다.

```
images_uris → texture_source_uris
```

02
Texture Load

상대 URI를 dds 파일 경로로 복원

base_path와 URI를 조합해 텍스처 로더가 읽을 경로로 변환했습니다.

```
base_path / uri → load_texture()
```

03
Shader Bind

Material 상수와 SRV 바인딩

factor 값과 텍스처 플래그를 넘겨 HLSL 샘플링 분기에 사용했습니다.

```
bind_material() → PS_GLTF()
```

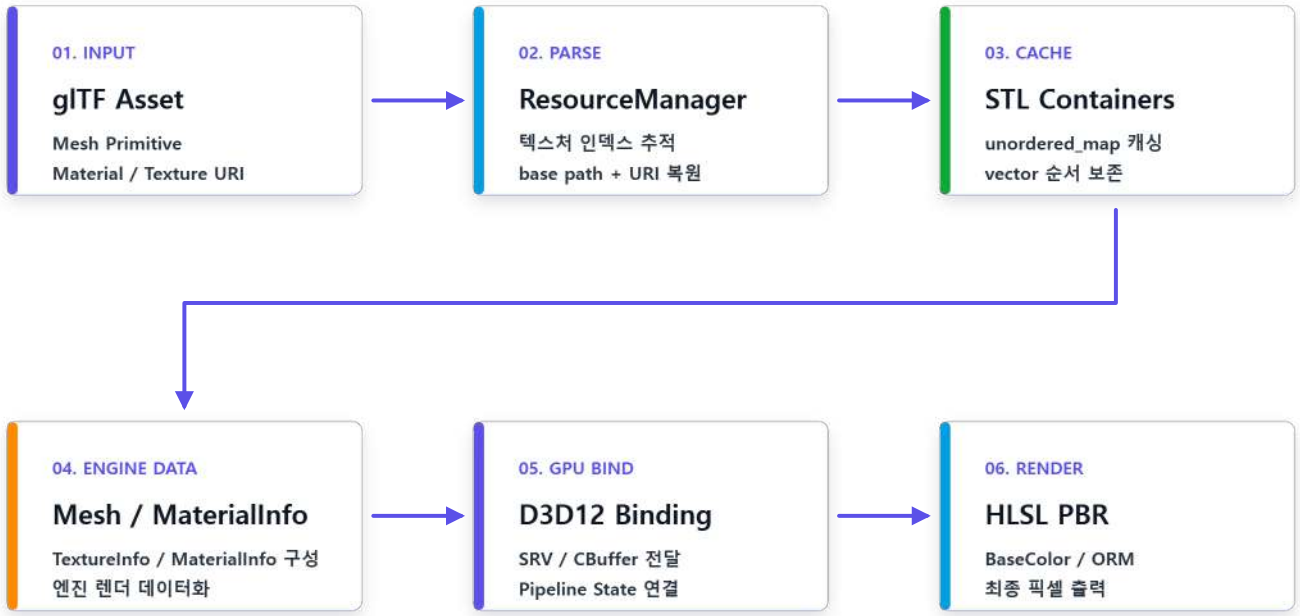
중요 코드 설명 1

C++20 · STL

리소스 전달 구조

Unreal에서 추출한 glTF의 images / textures / materials 데이터를 추적해 dds 텍스처 경로와 머티리얼 슬롯을 엔진 데이터로 변환했습니다.

1	**unordered_map 캐싱**	문자열 키 기반 캐시로 리소스 중복 로드 방지
2	**vector URI 배열**	glTF 인덱스 순서를 유지해 images와 textures 연결 순서 보존
3	**filesystem 경로 조합**	모델 기준 상대 URI를 실제 dds 경로로 복원



핵심 | 문자열 키 캐싱 → glTF 순서 보존 → 경로 복원 → 엔진 데이터 변환 → GPU 바인딩 → PBR 렌더링

glTF Asset → ResourceManager → 엔진 데이터 → D3D12 바인딩 → HLSL PBR

중요 코드 설명 2

HLSL

Shader 활용 포인트

ResourceManager가 채운 Material 상수와 텍스처 플래그를 HLSL에서 사용해 PBR 기본값과 ORM 샘플링 값을 같은 경로로 처리했습니다.

1	**CPU to GPU**	CPU MaterialInfo를 GPU cbMaterial에 매핑
2	**Texture Flag**	ORM 텍스처 유무로 factor와 샘플링 값을 분기
3	**Final Color**	Lighting / IBL / Emissive에 Phantom Aura를 합산

01. cbMaterial Structure

Gltf_Shader.hlsl

```
cbuffer cbMaterial : register(b2)
{
  float4 BaseColorFactor;
  float4 EmissiveAndMetallicFactor;
  #define EmissiveFactor EmissiveAndMetallicFactor.rgb
  #define MetallicFactor EmissiveAndMetallicFactor.a

  float RoughnessFactor;
  int HasMetallicRoughnessTexture;
  ....
};
```

02. Material Value Usage

Gltf_Shader.hlsl

```
roughness = lerp(RoughnessFactor, orm.g * RoughnessFactor, hasMetTex);
metallic = lerp(MetallicFactor, orm.b * MetallicFactor, hasMetTex);
float3 emissive = finalEmissive.rgb;

float4 litColor = Lighting(..., albedo, metallic, roughness, ao);
float3 iblColor = CalculateIBL(N, V, albedo, metallic, roughness, ao);
finalColor = (litColor.rgb * shadowFactor) + iblColor + finalEmissive;

if (g_otherplayerid > -1)
  finalColor = ApplyPhantomAura(finalColor, N, V);
```

개발 결과 및 성과

결과

- D3D12 기반 자체 렌더링 파이프라인 구성
- glTF 에셋을 실제 게임 화면에 안정적으로 출력
- BaseColor, Normal, ORM, Emissive 텍스처를 통합한 PBR 셰이더 구조 구현
- 인스턴싱·그림자 컬링으로 10FPS → 85FPS 이상 개선

성과

- 그래픽스 구현 내용을 논문 주제와 연결해 기술 구현 과정 문서화
- 한국게임학회 춘계학술발표대회 포스터 논문 발표



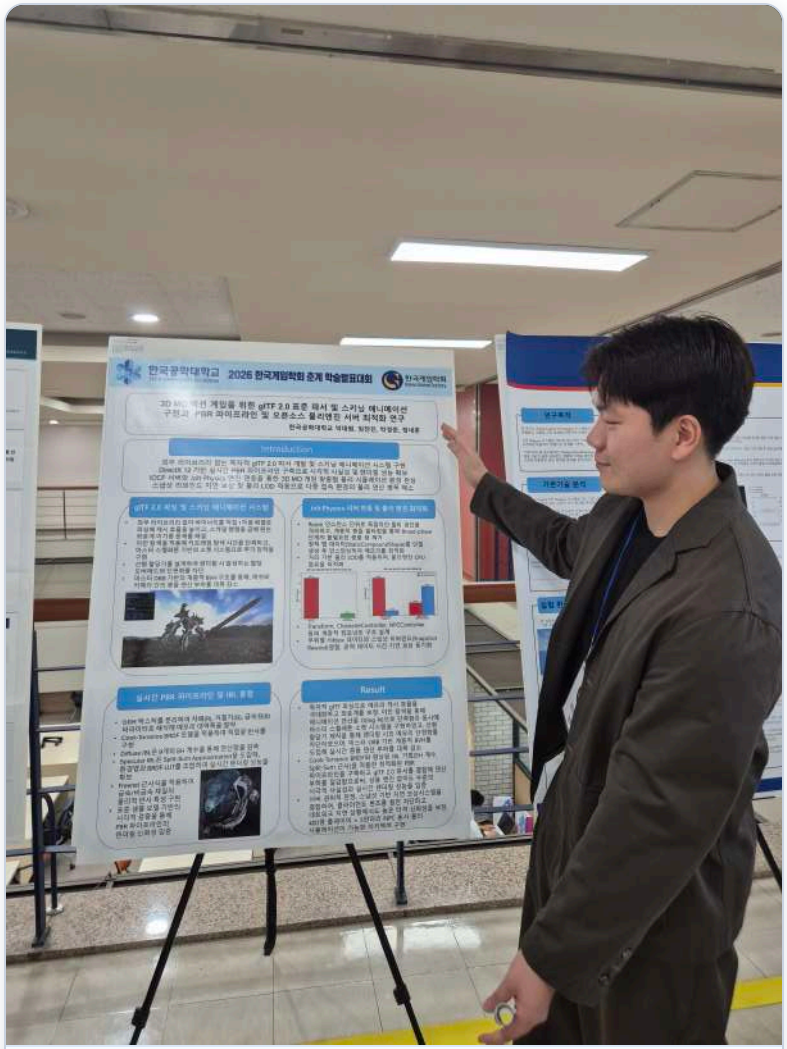
S.T.L 최종 렌더링 화면

졸업작품 우수작 선정

한국공학대학교 게임공학과 · 특성화장학금 수상

S.T.L에서 직접 구현한 glTF 파서, PBR 셰이더, D3D12 렌더링 파이프라인을 기술 주제로 정리해 다음 페이지의 논문 발표로 이어졌습니다.

논문 발표



2026 한국게임학회 춘계학술발표대회 포스터 발표

한국게임학회 춘계 학술발표대회

3D MO 액션 게임을 위한 glTF 2.0 표준 파서 및 스키닝 애니메이션 구현과 PBR 파이프라인 및 오픈소스 물리엔진 서버 최적화 연구

1 **담당 파트 · 그래픽스 구현** glTF 2.0 표준 파서와 PBR 파이프라인 구현 내용을 중심으로 정리했습니다.

2 **핵심 구현** 외부 라이브러리 의존을 줄이고 DirectX 12 환경에서 메시와 머티리얼 데이터를 직접 처리했습니다.

3 **PBR 파이프라인** ORM 텍스처, Cook-Torrance BRDF, IBL, Fresnel 기반 표현을 활용해 실시간 렌더링 품질을 개선했습니다.

S.T.L 후속 개발 / C++20 · DIRECTX 12

Forward에서 Deferred로: 렌더링 파이프라인 확장

GITHUB

Personal Project



PBR 직접광 + IBL 최종 렌더링 · Damaged Helmet(외부 샘플 에셋)

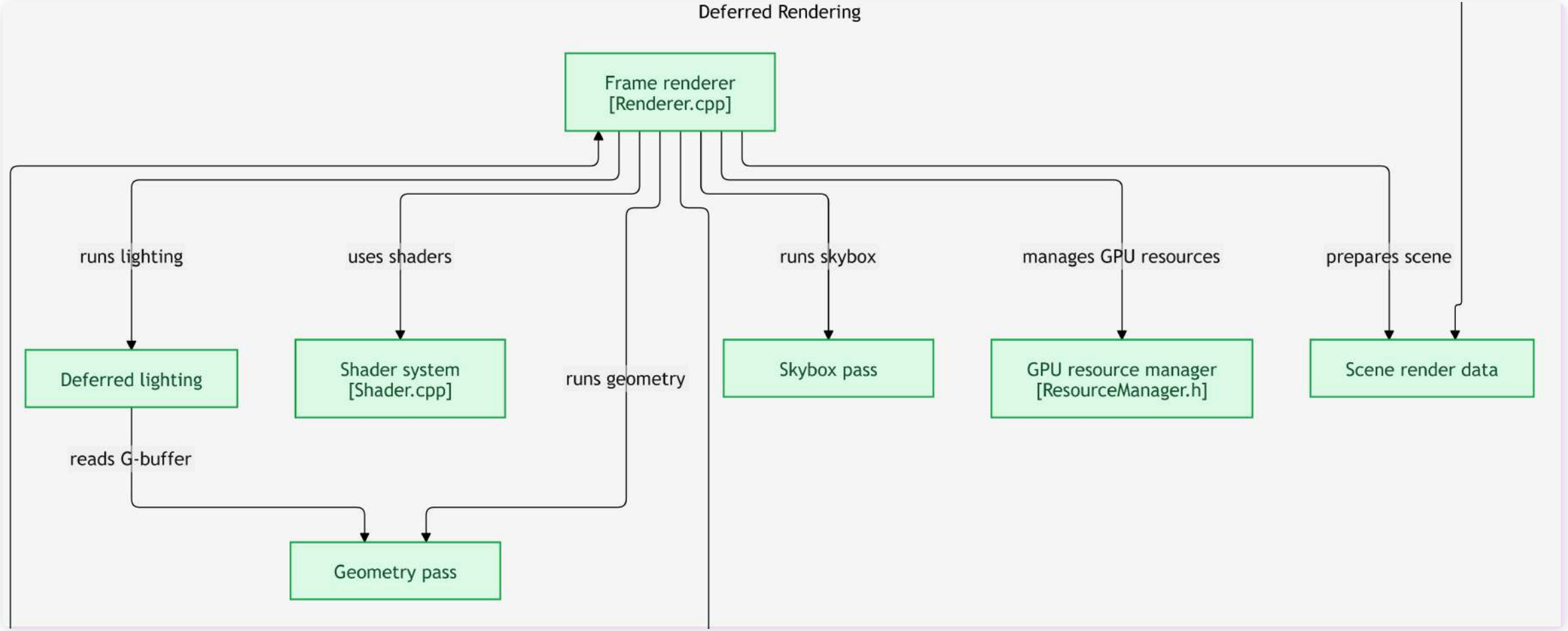
졸업작품 S.T.L의 Forward 렌더링 경험을 바탕으로, **표면 정보 생성과 조명 계산을 분리**하는 Deferred 파이프라인을 갖춘 PBR 메시 뷰어를 구현했습니다.

Geometry → **Skybox** → **Deferred Lighting** → **UI** → **Present**

Deferred 패스	Geometry · Skybox · Lighting · UI 패스 분리, 패스 간 GPU 리소스 상태 관리
G-buffer	5개 색상 타깃 + Depth, Full 40B / Compact 20B 전환과 깊이 기반 위치 복원
PBR · IBL	GGX · Schlick Fresnel · Smith Geometry, SH Diffuse · Prefiltered Specular · BRDF LUT
뷰어 · Import	런타임 glTF 불러오기, 객체 선택·변환, 표시 모드와 광원 조절 UI

원본 S.T.L 게임 전체를 Deferred로 전환한 것이 아니라, 그 경험을 바탕으로 만든 별도의 후속 렌더러입니다. 성능 우위보다 렌더 패스 설계와 데이터 전달 구조에 초점을 두었습니다.

렌더러 구조와 프레임 흐름



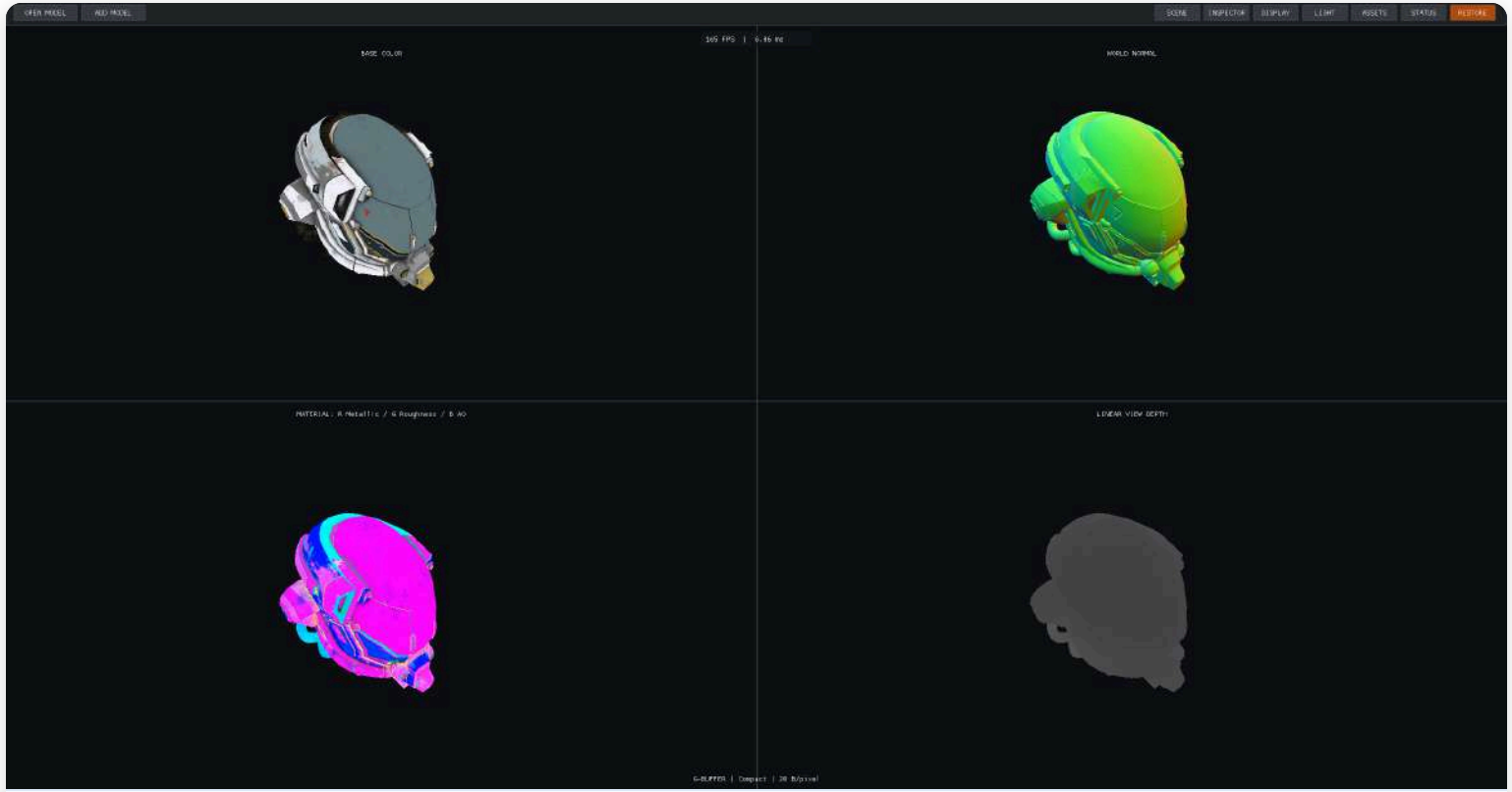
Renderer
초기화, 패스 실행 순서, 프레임 제출, 창 크기 변경

SceneRenderData
객체·재질 설정을 GPU 데이터와 Draw 목록으로 변환

GBufferRenderPass
G-buffer 타깃·RTV·PSO 관리와 메시 드로우

DeferredLightPass
G-buffer·IBL SRV를 연결해 전체 화면 조명 계산

G-buffer 구성과 저장 비용 개선



Geometry Pass 결과를 읽는 디버그 화면 · 좌상 Base Color · 우상 World Normal · 좌하 Material · 우하 Depth

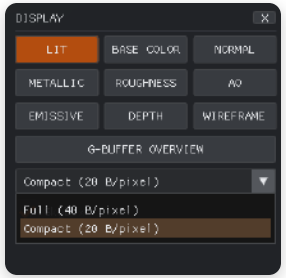
패스 간 리소스 관리

Geometry Pass 전에는 `RENDER_TARGET`, 이후에는 `PIXEL_SHADER_RESOURCE`로 상태를 전환하고, Lighting Pass가 `Texture2D.Load`로 같은 픽셀의 표면 정보를 읽습니다. 창 크기 변경 시 GPU 작업 완료를 기다린 뒤 타깃과 SRV를 다시 만듭니다.

저장 구조 · Full vs Compact

데이터	Full	Compact
Base Color	**4B**	**4B**
World Normal	**8B**	**4B**
Metal · Rough · AO	**4B**	**4B**
Emissive	**8B**	**4B**
월드 위치 / 깊이	**16B**	**4B**
합계 / pixel	**40B**	**20B**

위치 복원 Compact는 깊이만 기록하고, 화면 좌표와 깊이를 역 View-Projection 행렬로 변환한 뒤 W로 나눠 월드 위치를 복원합니다.



실행 중 전환

DISPLAY 패널에서 Lit·채널별 표시와 G-buffer Overview, Full / Compact 레이아웃을 바꿔 가며 확인했습니다. 기본값은 Compact입니다.

50% 감소는 5개 색상 타깃의 픽셀당 용량 기준이며, 전체 VRAM이나 GPU 시간 감소를 뜻하지 않습니다.

Forward vs Deferred 비교와 정리



같은 모델·같은 시점 · 가운데 선 기준 좌: Forward / 우: Deferred

같은 모델로 두 경로 비교

S.T.L의 Forward 렌더링을 옮긴 엔진과 새로 구현한 Deferred 렌더러로 같은 모델·같은 시점을 렌더링해 비교했습니다. Forward 쪽에는 S.T.L 게임 분위기에 맞춰 **IBL 강도를 낮춰 둔 설정**이 남아 있어 차이가 생겼습니다. 두 방식은 조명 계산 시점만 다르기 때문에, 조명 입력을 맞추면 원래 같은 결과가 나와야 합니다.

공부하며 정리한 선택 기준

Forward가 유리할 때

- 광원 수가 적고 장면이 단순할 때
- 투명·반투명 재질이 많을 때
- MSAA가 필요하거나 메모리 대역폭이 제한된 모바일 환경

Deferred가 유리할 때

- 동적 광원이 많아 조명 비용을 형상 수와 분리해야 할 때
- 불투명 물체 위주의 장면
- G-buffer를 SSAO·SSR 같은 화면 공간 효과에 재활용할 때

두 방식을 직접 구현해 보며 위와 같이 정리했고, 상황에 맞는 방식을 고르는 기준으로 삼고 있습니다.

라이팅 수치에 대해 배운 점

S.T.L에서 IBL을 낮춰 분위기를 맞췄던 경험을 다시 보며, 물리적으로 정확한 값이 항상 게임에 맞는 값은 아니라는 것을 느꼈습니다. **실제 물리 기준이 필요한 경우가 아니라면** 게임 분위기를 위해 라이팅 수치를 타협할 수 있다고 생각하게 되었고, 검증용 기준 값과 연출용 값을 구분해서 보게 되었습니다.

TheGreatestNature

GITHUB VIDEO

Team Project









MetaHuman
리타게팅과 표정 제어

Sequencer
Camera Rail과 컷 편집

조명
장면 분위기에 맞춘 조명 구성

단편 시네마틱 영상

TheGreatestNature

자연을 바라보며 치유되는 감정을 MetaHuman과 Sequencer로 연출한 UE5 시네마틱

개발 인원	2명
담당 역할	MetaHuman 리타게팅, 표정 제어, Camera Rail, Sequencer 컷 편집, 조명 구성
개발 언어	Blueprint
사용 툴	Unreal Engine 5, MetaHuman, Sequencer, Git
제작 기간	2025.11.11 ~ 2025.12.16

ASSASSIN VS MUTANT

GITHUB VIDEO

Personal Project









적 AI

Behavior Tree · Blackboard로 행동 제어

구조 설계

Event Dispatcher · Interface 통신

전투 흐름

무기 스위칭 · UI · 승패 흐름

3인칭 3D 액션 게임

ASSASSIN VS MUTANT

암살자 캐릭터가 돌연변이 적을 상대하는 UE5 기반 액션 전투 프로젝트

개발 인원	1명
담당 역할	전투 액션, 무기 스위칭, 적 AI, UI, 승패 흐름, 연출 이미지 적용
개발 언어	Blueprint
사용 툴	Unreal Engine 5, Git
제작 기간	2025.06.01 ~ 2025.06.18

FlickDom (AI VIBE CODING)

GITHUB VIDEO

Team Project





목표 칸 점령 · 카드 패턴 완성



조작 드래그 방향·힘으로 발사



멀티 Host 방 생성 · Join Code

1 VS 1 전략 플릭 보드게임 · AI VIBE CODING

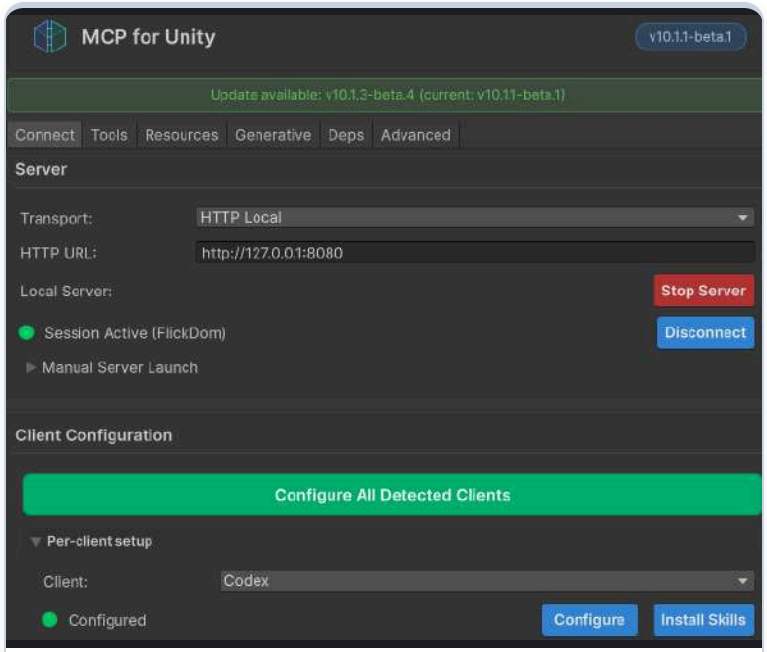
FlickDom

디스크를 튕겨 보드 칸을 차지하고 카드 패턴을 완성해 점수를 겨루는 물리 기반 파티 게임

개발 인원	3명 (모델러 1 · 구현 1 · 본인)
담당 역할	게임 구현 및 기획 총괄, AI 작업용 MD 파일 구조화, 멀티플레이 흐름, 사운드 제작 및 적용
개발 언어	C#
사용 툴	Unity 6 Editor, VS Code, Blender
AI 모델	GPT-5 Codex, Unity MCP, Blender MCP, VARCO AI Sound
제작 기간	2026.07.01 ~ 2026.08.10

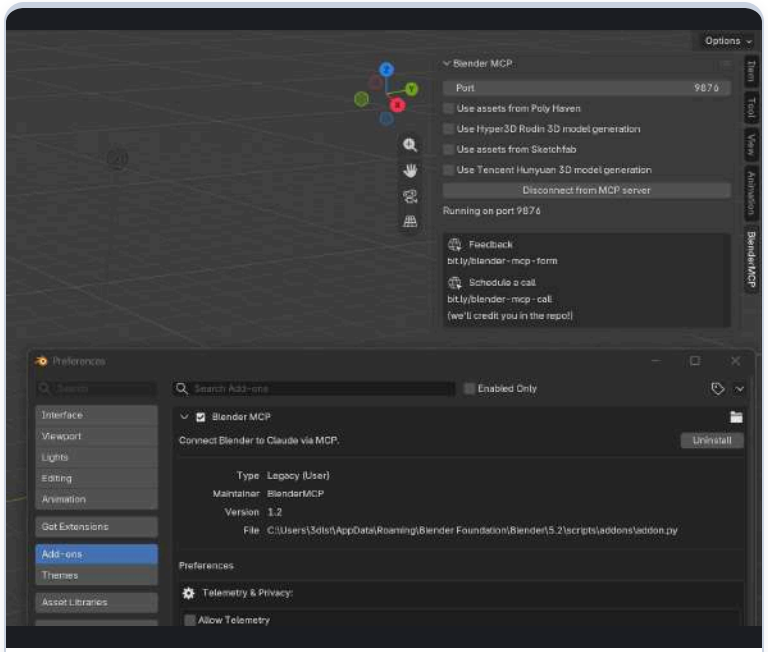
AI/MCP 기반 개발

AI 에이전트(MCP)를 엔진·툴과 직접 연결해 구현 반복과 리소스 제작 속도를 높였습니다.



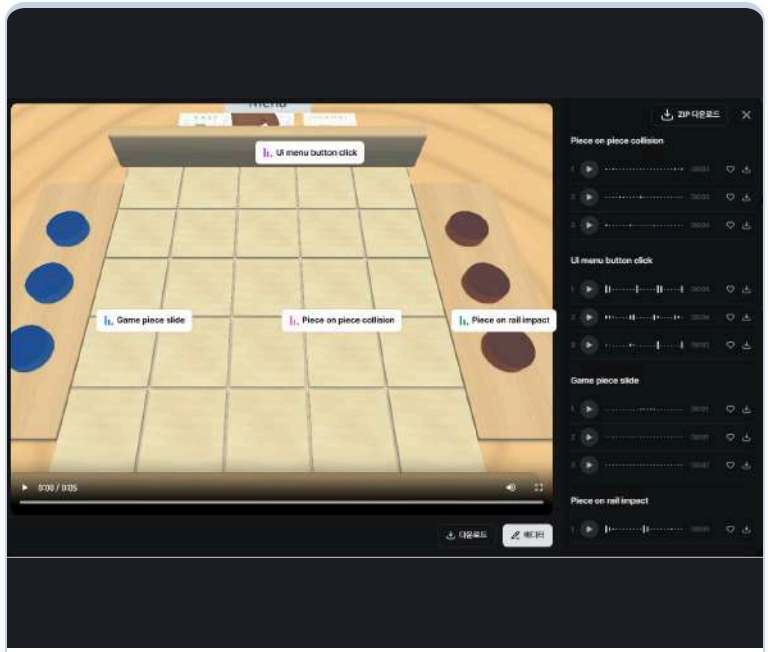
Unity MCP

Unity Editor와 AI 에이전트를 연결해 씬, 오브젝트, 컴포넌트 작업을 빠르게 확인하고 반복했습니다.



Blender MCP

보드, 디스크, 소품 리소스의 비율과 배치 목적을 정리하며 3D 제작 과정을 보조했습니다.



VARCO AI Sound

버튼 클릭, 점수 획득, 실패, 승리 등 상황별 효과음을 제작해 플레이어 피드백을 강화했습니다.

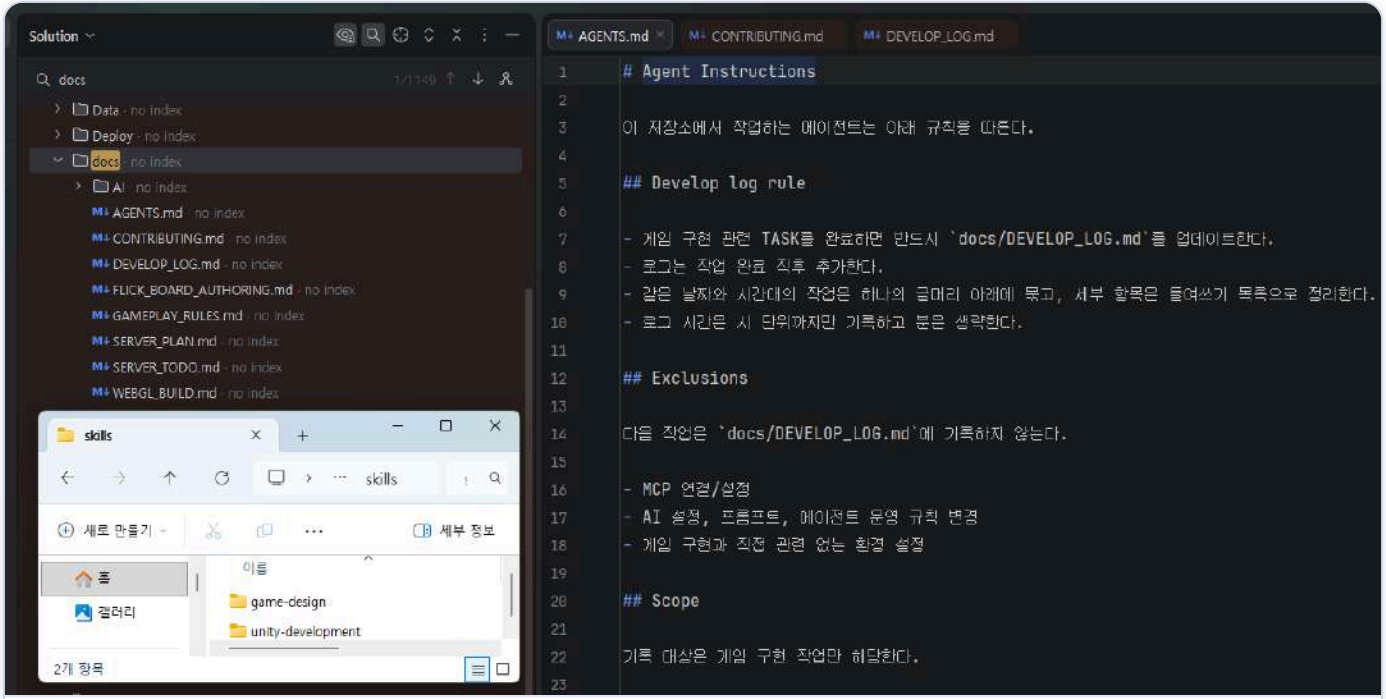
MD 기반 프로젝트 뼈대

FlickDom

MD 기반 프로젝트 뼈대 및 모듈화

Architecture.md에 실제 프로젝트 구조와 작업 규칙을 정리하고, 기획·구현·빌드 문서를 기능별로 분리해 AI(MCP)가 작업 맥락을 빠르게 파악하도록 구성했습니다.

1	**작업 기준**	Architecture.md에 FlickDom의 실제 폴더 구조와 시스템 기준 정리
2	**기획 모듈**	GAMEPLAY_RULES.md로 핵심 규칙, SERVER_PLAN.md로 동기화 계획 분리
3	**제작 및 빌드 기준**	리소스 제작 기준과 WebGL 배포 절차를 별도 문서로 관리



FlickDom docs 폴더의 기능별 MD 문서와 AGENTS.md 작업 규칙

주요 문서

`AGENTS.md` `CONTRIBUTING.md` `DEVELOP_LOG.md` `GAMEPLAY_RULES.md` `SERVER_PLAN.md`
`SERVER_TODO.md` `FLICK_BOARD_AUTHORING.md` `WEBGL_BUILD.md`

게임 제작 기본 MD 파일

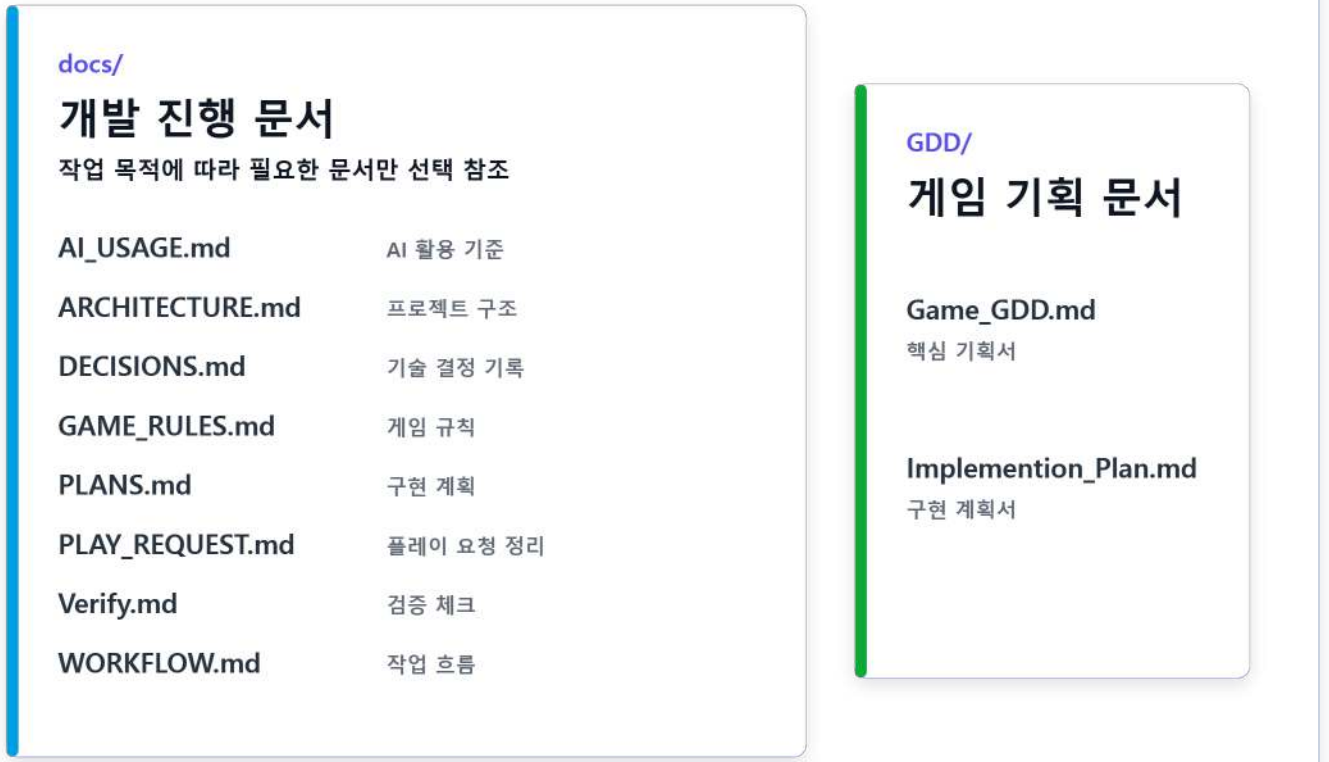
GITHUB

BASE_MD

새로운 MD 참조 구조

FlickDom에서 MD 간 연결이 강해 작업마다 참조 범위와 AI 토큰 사용이 커지는 문제를 겪고, 요청 목적에 맞는 하위 문서만 선택해 읽는 구조로 바꿨습니다.

1	**문제 인식**	MD 간 연결이 강할수록 작업별 참조 범위가 과도하게 확장
2	**읽기 방식 수정**	요청 목적에 맞춰 docs/, GDD/ 하위 파일 중 필요한 문서만 선택
3	**다음 프로젝트 적용**	기획, 구현, 검증 문서를 목적별로 분리해 읽는 방식으로 사용 예정



docs/ 개발 진행 문서와 GDD/ 게임 기획 문서를 목적별로 분리

CONTACT

감사합니다.

이름	임찬진
전화번호	010-4032-9600
이메일	ckswlsdl0916@gmail.com